

Université Sultan Moulay Slimane
ENSA Beni Mellal
Département Intelligence artificielle et cybersécurité

Rapport de TP N°2
Restructuration en Monolithe Modulaire
avec MapStruct

Module : Architecture des Systèmes Distribués

Réalisé par : Rayhana Laznasni

Encadré par : [M.Elbaghazaoui]

Année universitaire : 2025 – 2026

Table des matières

1	Introduction	2
2	Objectifs du TP	2
3	Technologies utilisées	2
4	Architecture du projet	2
4.1	Structure des modules	2
4.2	Diagramme des dépendances	3
5	Étapes de réalisation	4
5.1	Étape 1 : Configuration de MapStruct	4
5.2	Étape 2 : Création des DTOs	5
5.3	Étape 3 : Création du Mapper	5
5.4	Étape 4 : Refactoring du Service	6
5.5	Étape 5 : Adaptation du Controller	6
5.6	Étape 6 : Module Customer	7
5.7	Étape 7 : Module Order	7
6	Endpoints de l'API	8
7	Concepts clés appris	8
8	Difficultés rencontrées	8
9	Conclusion	9
10	Références	9

1 Introduction

Dans le cadre du module **Architecture des Systèmes Distribués**, ce deuxième TP a pour objectif de transformer l'application monolithique simple développée lors du TP1 en un **monolithe modulaire** bien structuré.

L'idée principale est de préparer l'application à une future migration vers une architecture microservices, en appliquant les principes suivants :

- Séparation en modules indépendants (Product, Customer, Order)
- Introduction du pattern **DTO** (Data Transfer Object)
- Utilisation de **MapStruct** pour le mapping automatique
- Application du principe **Interface / Implémentation**
- Assurance d'un **couplage faible** entre les modules

2 Objectifs du TP

Les objectifs de ce TP sont les suivants :

1. **Restructurer** le code en modules métier indépendants
2. **Introduire les DTOs** pour séparer la couche de présentation de la couche de persistance
3. **Configurer MapStruct** pour automatiser le mapping entre entités et DTOs
4. **Refactorer les services** avec le pattern Interface/Implémentation
5. **Créer de nouveaux modules** (Customer et Order) en suivant la même architecture
6. **Gérer les dépendances** inter-modules via les interfaces de service uniquement

3 Technologies utilisées

Technologie	Version	Rôle
Java	17+	Langage de programmation principal
Spring Boot	3.x	Framework backend
Spring Data JPA	–	Accès aux données et ORM
MapStruct	1.5.5	Mapping automatique Entity ↔ DTO
Lombok	–	Réduction du code boilerplate
H2 Database	–	Base de données en mémoire pour le développement
Maven	–	Gestion des dépendances et build

TABLE 1 – Technologies utilisées dans le projet

4 Architecture du projet

4.1 Structure des modules

Le projet est organisé en trois modules métier indépendants :

Listing 1 – Structure du projet

```
1 com.ecommerce.monolith
2 |-- product/                (Module independant)
3 |   |-- controller/
4 |   |-- dto/
5 |   |-- mapper/
6 |   |-- model/
7 |   |-- repository/
8 |   +-- service/
9 |
10 |-- customer/              (Module independant)
11 |   |-- controller/
12 |   |-- dto/
13 |   |-- mapper/
14 |   |-- model/
15 |   |-- repository/
16 |   +-- service/
17 |
18 |-- order/                 (Depend de Product & Customer)
19 |   |-- controller/
20 |   |-- dto/
21 |   |-- mapper/
22 |   |-- model/
23 |   |-- repository/
24 |   +-- service/
25 |
26 +-- MonolithApplication.java
```

4.2 Diagramme des dépendances

Le module **Order** dépend des modules **Product** et **Customer**, mais uniquement via leurs **interfaces de service**. Les modules **Product** et **Customer** sont totalement indépendants l'un de l'autre.

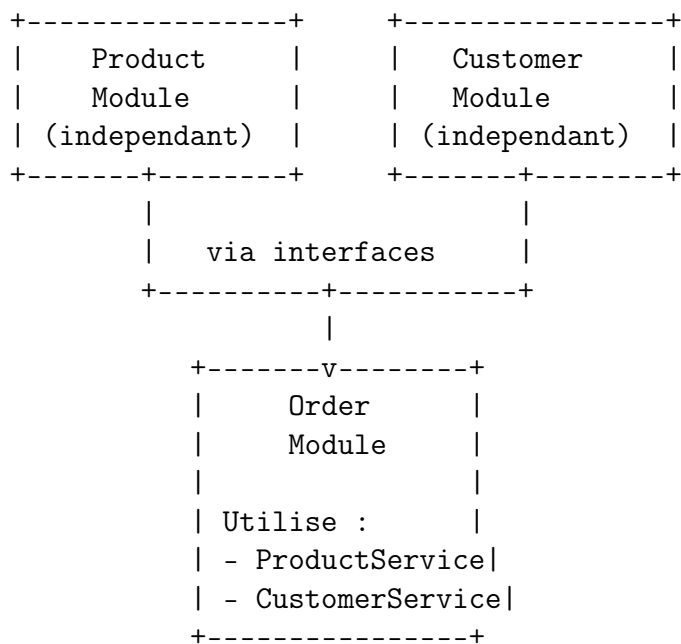


FIGURE 1 – Diagramme des dépendances entre modules

5 Étapes de réalisation

5.1 Étape 1 : Configuration de MapStruct

La première étape consiste à ajouter les dépendances MapStruct dans le fichier `pom.xml` :

Listing 2 – Dépendances MapStruct dans pom.xml

```
1 <dependency>
2   <groupId>org.mapstruct</groupId>
3   <artifactId>mapstruct</artifactId>
4   <version>1.5.5.Final</version>
5 </dependency>
```

Et la configuration du plugin de compilation avec le processeur d'annotations :

Listing 3 – Configuration du plugin Maven

```

1 <plugin>
2   <groupId>org.apache.maven.plugins</groupId>
3   <artifactId>maven-compiler-plugin</artifactId>
4   <configuration>
5     <annotationProcessorPaths>
6       <path>
7         <groupId>org.mapstruct</groupId>
8         <artifactId>mapstruct-processor</artifactId>
9         <version>1.5.5.Final</version>
10      </path>
11      <path>
12        <groupId>org.projectlombok</groupId>

```

```
13         <artifactId>lombok</artifactId>
14     </path>
15     <path>
16         <groupId>org.projectlombok</groupId>
17         <artifactId>lombok-mapstruct-binding</artifactId>
18         <version>0.2.0</version>
19     </path>
20 </annotationProcessorPaths>
21 </configuration>
22 </plugin>
```

5.2 Étape 2 : Création des DTOs

Les DTOs permettent de séparer les données exposées via l'API des entités JPA stockées en base de données.

Exemple : ProductDTO.java

Listing 4 – ProductDTO.java

```
1 @Data
2 public class ProductDTO {
3     private Long id;
4     private String name;
5     private String description;
6     private BigDecimal price;
7     private Integer stock;
8 }
```

Exemple : CreateProductRequest.java

Listing 5 – CreateProductRequest.java

```
1 @Data
2 public class CreateProductRequest {
3     @NotBlank(message = "Product name is required")
4     private String name;
5     private String description;
6     @NotNull @Positive
7     private BigDecimal price;
8     @NotNull @PositiveOrZero
9     private Integer stock;
10 }
```

5.3 Étape 3 : Création du Mapper

MapStruct génère automatiquement le code de conversion entre les entités et les DTOs :

Listing 6 – ProductMapper.java

```
1 @Mapper(componentModel = "spring")
2 public interface ProductMapper {
3     ProductDTO toDTO(Product product);
```

```
4 List<ProductDTO> toDTOList(List<Product> products);
5 Product toEntity(CreateProductRequest request);
6
7 @Mapping(target = "id", ignore = true)
8 void updateEntity(CreateProductRequest request,
9                  @MappingTarget Product product);
10 }
```

5.4 Étape 4 : Refactoring du Service

Le service est séparé en une **interface** (contrat) et une **implémentation** :

Listing 7 – ProductService.java (Interface)

```
1 public interface ProductService {
2     List<ProductDTO> getAllProducts();
3     ProductDTO getProductById(Long id);
4     ProductDTO createProduct(CreateProductRequest request);
5     ProductDTO updateProduct(Long id, CreateProductRequest request)
6         ;
7     void deleteProduct(Long id);
8 }
```

5.5 Étape 5 : Adaptation du Controller

Le controller utilise désormais les DTOs au lieu des entités :

Listing 8 – ProductController.java

```
1 @RestController
2 @RequestMapping("/api/products")
3 @RequiredArgsConstructor
4 public class ProductController {
5     private final ProductService productService;
6
7     @GetMapping
8     public ResponseEntity<List<ProductDTO>> getAll() {
9         return ResponseEntity.ok(productService.getAllProducts());
10    }
11
12    @PostMapping
13    public ResponseEntity<ProductDTO> create(
14        @Valid @RequestBody CreateProductRequest request) {
15        ProductDTO created = productService.createProduct(request);
16        return ResponseEntity.status(HttpStatus.CREATED).body(
17            created);
18    }
19 }
```

5.6 Étape 6 : Module Customer

Le module Customer suit exactement la même architecture que le module Product :

- **Entity** : Customer.java avec les champs name, email, phone, address
- **DTOs** : CustomerDTO et CreateCustomerRequest
- **Mapper** : CustomerMapper avec MapStruct
- **Service** : CustomerService (interface) + CustomerServiceImpl
- **Controller** : CustomerController avec les endpoints CRUD

Ce module est **totalelement indépendant** et n'a aucune dépendance vers les autres modules.

5.7 Étape 7 : Module Order

Le module Order est le plus intéressant car il **dépend** des modules Product et Customer, mais uniquement via leurs **interfaces de service** :

Listing 9 – Dépendances du OrderServiceImpl

```
1 @Service
2 @RequiredArgsConstructor
3 public class OrderServiceImpl implements OrderService {
4     private final OrderRepository orderRepository;
5     private final OrderMapper orderMapper;
6     private final ProductService productService;    // Interface
7     private final CustomerService customerService; // Interface
8 }
```

Point clé : Le module Order n'accède **jamais** directement aux repositories des autres modules. Il passe toujours par les interfaces de service, garantissant ainsi un couplage faible.

6 Endpoints de l'API

Méthode	URL	Description
Products (/api/products)		
GET	/api/products	Liste tous les produits
GET	/api/products/{id}	Récupère un produit par ID
POST	/api/products	Crée un nouveau produit
PUT	/api/products/{id}	Met à jour un produit
DELETE	/api/products/{id}	Supprime un produit
Customers (/api/customers)		
GET	/api/customers	Liste tous les clients
GET	/api/customers/{id}	Récupère un client par ID
POST	/api/customers	Crée un nouveau client
PUT	/api/customers/{id}	Met à jour un client
DELETE	/api/customers/{id}	Supprime un client
Orders (/api/orders)		
GET	/api/orders	Liste toutes les commandes
GET	/api/orders/{id}	Récupère une commande par ID
POST	/api/orders	Crée une nouvelle commande
DELETE	/api/orders/{id}	Supprime une commande

TABLE 2 – Liste des endpoints de l'API REST

7 Concepts clés appris

Concept	Explication
DTO	Séparer les données exposées en API de celles stockées en BDD
MapStruct	Génération automatique du code de conversion Entity ↔ DTO
Interface / Impl	Découpler le contrat du service de son implémentation
Couplage faible	Les modules dépendent des interfaces, jamais des implémentations
Modularité	Chaque module métier est autonome et indépendant

TABLE 3 – Concepts clés du TP2

8 Difficultés rencontrées

Durant la réalisation de ce TP, plusieurs difficultés ont été rencontrées :

1. **Configuration de MapStruct avec Lombok** : L'ordre des processeurs d'annotations est important. Il faut déclarer le `lombok-mapstruct-binding` pour que les deux bibliothèques fonctionnent ensemble.
2. **Résolution des symboles** : Certaines erreurs de compilation comme `Cannot resolve symbol 'ProductMapper'` étaient dues à des packages manquants ou mal nommés.

3. **Fautes de frappe** : Un fichier nommé `CustommerServiceImpl` (double m) au lieu de `CustomerServiceImpl` a causé des erreurs de résolution.
4. **Adaptation du Controller** : Le passage des entités aux DTOs dans le controller a nécessité la mise à jour de tous les types de retour et paramètres.

9 Conclusion

Ce TP nous a permis de transformer une application monolithique simple en un **monolithe modulaire** bien structuré. Les principaux acquis sont :

- La maîtrise du pattern **DTO** et de **MapStruct**
- La compréhension du principe de **couplage faible** entre modules
- L'application du pattern **Interface/Implémentation** pour les services
- La préparation de l'application pour une future migration vers une **architecture microservices**

Cette restructuration facilite la maintenance, les tests, et prépare le terrain pour le **TP3** où l'on passera à une architecture distribuée avec des microservices.

10 Références

- Documentation officielle de Spring Boot : <https://spring.io/projects/spring-boot>
- Documentation MapStruct : <https://mapstruct.org/documentation/stable/reference/html/>
- Documentation Lombok : <https://projectlombok.org/features/>
- Repository GitHub du projet : https://github.com/Neriona/TP-Java-SpringBoot-Rayhana_laznasni